

Aspect	Region Pinning for G1
Java Version	Java 22
Year	2024
Feature	Region Pinning for G1
UniqueClassName	RegionPinningG1
What it is	An improvement to the G1 garbage collector that allows certain G1 regions containing objects that cannot be moved safely at a particular time to be pinned , preventing G1 from relocating those objects during garbage collection.
Primary goal	Improve G1's ability to handle objects that need stable memory locations, particularly during operations involving JNI and critical native-memory access.
Main benefit	Reduces problems caused by objects that must temporarily remain at a fixed location while allowing G1 to continue performing its normal garbage-collection work.
How it works — simple view	Normally, G1 can evacuate objects from one region to another. With region pinning, a region can temporarily be marked as pinned , so G1 does not evacuate its objects until the pinning condition ends.
Advantages	1. Better support for native/JNI interactions. 2. Prevents unsafe object movement while native code is using an object's address. 3. Allows G1 to continue using its region-based collection model. 4. Reduces the need for overly conservative GC behavior in applications using native code. 5. Particularly useful for applications that combine Java and native libraries.
Disadvantages	1. Pinned regions cannot be evacuated normally while pinned. 2. Excessive pinning can reduce G1's freedom to compact/evacuate the heap. 3. May contribute to fragmentation if many regions remain pinned for long periods. 4. Primarily benefits workloads involving the relevant native/JNI access patterns; ordinary Java applications may see little direct benefit. 5. Application developers generally cannot treat region pinning as a normal application-level feature to manually control.
When to use	Use G1 on Java 22 when your application uses JNI/native libraries

	or other operations where Java objects need to remain safely accessible at a stable location for a period of time.
When not to use	Don't choose G1 specifically <i>because</i> of region pinning if your application is a conventional Java application with no relevant native/JNI requirements. Select the GC based on the application's overall latency, throughput, heap size, and allocation characteristics.
Real-time use case 1	Java + native image/video processing: Java passes data to a native library through JNI. Region pinning can help when the native operation requires the corresponding Java memory to remain stable during the operation.
Real-time use case 2	High-performance database driver: A Java application communicates with a database through native components. Stable access to Java memory can be important during native operations.
Real-time use case 3	Scientific/engineering application: Java performs computation while native numerical libraries process Java-provided data. Pinning helps accommodate native access without requiring G1 to freely relocate the relevant objects during the critical operation.
Real-time use case 4	Operating-system integration: Java code interacts with native OS APIs through JNI where native code temporarily accesses Java-managed data.
Example	Imagine a Java application passing a byte buffer to native code. Normally, G1 may move objects during evacuation. If the object must remain at a stable location while native code is accessing it, region pinning allows the relevant G1 region to remain in place until the operation is complete.
Alternate feature	ZGC can be considered when the primary requirement is extremely low GC pause times and large heaps rather than G1's region-based evacuation behavior. For applications without native-memory constraints, ordinary G1 operation may itself be the appropriate alternative.
Best suited for	Large Java applications using G1 + JNI/native code , especially when native operations temporarily require stable access to Java-managed objects.
Not a replacement for	It is not a replacement for G1, ZGC, or another garbage collector . It is an improvement to G1's handling of pinned regions.

G1 Region Pinning vs Alternatives

Requirement	Region Pinning for G1	Normal G1	ZGC
Primary purpose	Safely handle regions that temporarily cannot be evacuated	General-purpose low-pause GC	Extremely low-latency garbage collection
Java 22	Yes	Yes	Yes
Region-based design	Yes	Yes	Different region/page architecture
Native/JNI scenarios	Particularly relevant	Can be problematic when objects need stable locations	Designed around concurrent relocation mechanisms
Object relocation	Temporarily prevents relocation of pinned regions	Frequently evacuates objects	Concurrent relocation
Typical application	Java/native integration	General server applications	Very large heaps and latency-sensitive systems
Main advantage	Handles temporary object-location constraints within G1	Balanced throughput and latency	Very low pause times
Main drawback	Pinned regions can constrain evacuation	Can encounter limitations around pinned objects	Can have higher complexity/resource overhead
Use when	You specifically need G1 to accommodate temporarily pinned regions	You need a balanced general-purpose collector	Ultra-low pause latency is the dominant requirement

In short: Region Pinning for G1 is best viewed as a **G1 robustness improvement for workloads involving objects that temporarily cannot be moved**, rather than a feature you directly enable for ordinary application development.

Aspect	Statements before super(...) (Preview)
Java Version	Java 22
Year	2024
Feature	Statements before super(...) — Preview
UniqueClassName	StatementsBeforeSuper
JEP	JEP 447 — Statements before super(...) (Preview)
What it is	Allows a constructor to execute certain statements before the explicit or implicit super(...) constructor invocation . This makes it possible to perform validation, prepare arguments, or calculate values before invoking the superclass constructor.
Problem it solves	Traditionally, a constructor had to invoke super(...) or this(...) before executing ordinary statements. This could make constructor logic unnecessarily difficult when values needed by the superclass had to be calculated or validated first.
Syntax idea	<pre>```javaclass Employee extends Person { Employee(String name) { if (name == null</pre>
Important restriction	Statements before super(...) cannot arbitrarily access the newly created object. In particular, they cannot use this or access instance members in ways that would depend on the object before superclass construction.
Advantages	1. Better constructor validation: Validate parameters before entering the superclass constructor. 2. Easier argument preparation: Calculate or transform arguments before passing them to super(...). 3. Cleaner inheritance code: Avoid awkward static helper methods solely to prepare superclass arguments. 4. Better readability: Validation and preparation can appear directly in the constructor. 5. More expressive constructors: Complex initialization sequences become easier to represent.

Disadvantages	1. Preview feature: In Java 22 it is a preview feature and requires preview compilation/runtime options. 2. Possible confusion: Developers accustomed to <code>super(...)</code> being first may initially find the syntax surprising. 3. Restricted access: Code before <code>super(...)</code> cannot freely use instance state because the object has not yet been initialized. 4. Constructor complexity: Too much logic before <code>super(...)</code> can make constructors difficult to understand. 5. Compatibility risk: Preview syntax may change before becoming final, so it should be used cautiously in production libraries.
When to use	Use it when constructor parameters must be validated, normalized, transformed, or otherwise prepared before calling the superclass constructor.
When not to use	Avoid it for simple constructors where <code>super(...)</code> can naturally be the first statement. Also avoid putting large business workflows inside constructors merely because the feature permits statements before <code>super(...)</code> .
Real-time use case — validation	A subclass can validate an identifier before passing it to the superclass: <pre>java class Customer extends Person { Customer(String id) { if (id == null</pre>
Real-time use case — data transformation	Convert an input value before the superclass receives it: <pre>java class PremiumCustomer extends Customer { PremiumCustomer(String email) { String normalized = email.trim().toLowerCase(); super(normalized); } }</pre>
Real-time use case — configuration	Calculate a configuration value before passing it to a framework/base class: <pre>java class ProductionService extends Service { ProductionService(Config config) { Config prepared = config.withDefaults(); super(prepared); } }</pre>
Real-time use case — security validation	Validate a role, permission, or identifier before allowing the superclass to initialize its state with that value.
Real-time use case — derived constructor argument	A subclass can derive a value from several constructor parameters and pass the resulting value to <code>super(...)</code> , without hiding the preparation logic in a separate helper method.
Alternate feature — <code>super(...)</code> directly	For simple constructors, keep the traditional approach: <pre>java class Employee extends Person { Employee(String name) { super(name); } }</pre>
Alternate feature —	If preparation is complex and reusable, calculate the argument

static helper method	through a static method: <pre>``java class Employee extends Person { Employee(String name) { super(validateAndNormalize(name)); } private static String validateAndNormalize(String name) { if (name == null</pre>
Alternate feature — factory method	If construction requires substantial preparation, a static factory such as <code>create(...)</code> can perform the preparation before invoking the constructor.
Best suited for	Subclass constructors requiring small amounts of validation or argument preparation before superclass construction.
Not ideal for	Large initialization workflows, database calls, network operations, business processes, or complicated object construction.
Status	Preview in Java 22. It was intended to make constructor initialization more flexible while preserving Java's safety guarantees around superclass construction.

Statements Before `super(...)` vs Alternatives

Scenario	Statements Before <code>super(...)</code>	Traditional <code>super(...)</code>	Static Helper	Factory Method
Simple constructor	☐ Unnecessary	✓ Best	✗ Overkill	✗ Overkill
Parameter validation	✓ Excellent	☐ Awkward	✓ Good	✓ Good
Parameter normalization	✓ Excellent	☐ Helper often needed	✓ Good	✓ Good
Complex preparation	☐ Use carefully	✗ Awkward	✓ Good	✓ Very good
Reusable preparation logic	☐ Possible	✗	✓ Excellent	✓ Excellent
Large business workflow	✗ Avoid	✗ Avoid	☐ Better	✓ Best
Java 22 preview	✓ Yes	✓ Yes	✓ Yes	✓ Yes

compatibility acceptable				
Public library API	☐ Be cautious	☒ Safest	☒ Safe	☒ Safe
Readability for small validation	☒ Excellent	☐ Less direct	☐ More indirection	☐ More indirection

Key takeaway

Statements before `super(...)` is primarily a **constructor flexibility feature**. Its biggest value is allowing a subclass constructor to do **small, safe pieces of validation or argument preparation before superclass construction**.

For example:

```
class Employee extends Person {
    Employee(String name) {
        String normalized = name.trim();

        if (normalized.isEmpty()) {
            throw new IllegalArgumentException("Name required");
        }

        super(normalized);
    }
}
```

Without this feature, such preparation traditionally had to be moved into a helper method or performed inside the superclass.

Recommendation: Use it when the pre-`super(...)` logic is **short, local, and directly related to preparing constructor arguments**. For complex construction logic, prefer a **factory method or dedicated helper**.

Aspect	Foreign Function & Memory API (FFM API)
Java Version	Java 22
Year	2024

Feature	Foreign Function & Memory API
UniqueClassName	ForeignFunctionMemoryAPI
JEP	JEP 454
Status in Java 22	Final — not a preview feature. It had been developed through earlier preview/incubator versions and was finalized in Java 22. (Oracle Docs)
Package	java.lang.foreign
Purpose	Allows Java applications to call native/foreign functions and safely work with memory outside the JVM heap , without relying on the traditional JNI approach. (Oracle Docs)
Main abstractions	MemorySegment — represents a region of memory; Arena — manages the lifetime of allocated memory; MemoryLayout — describes the structure/layout of memory; Linker — links Java code to foreign functions; SymbolLookup — finds native functions; FunctionDescriptor — describes a native function signature. (Oracle)
Advantages	1. Safer alternative to JNI: Provides a structured Java API for native interoperability. 2. No handwritten JNI boilerplate: Developers can invoke native functions directly through the FFM API. 3. Foreign-memory access: Provides controlled access to off-heap/native memory. 4. Deterministic memory lifetime: Arena provides explicit lifetime management for native memory. 5. Good performance: Designed to provide native calls with performance comparable to, and potentially better than, JNI in suitable scenarios. 6. Platform portability: The linker handles the native calling convention for the platform. 7. Better readability: Native interaction can remain largely in Java rather than requiring Java + C/C++ JNI glue code. (Oracle Docs)
Disadvantages	1. Native code is still unsafe: Incorrect use can crash the JVM or cause memory corruption. 2. More complex than ordinary Java: Developers need to understand memory layouts, segments, arenas, pointers, and native function signatures. 3. Platform dependencies remain: The native library itself must exist for the target platform/architecture. 4. Manual lifetime considerations: Although safer than many

	low-level alternatives, native memory still requires appropriate lifetime management. 5. Restricted operations: Some FFM operations are restricted and require native-access enablement. 6. Harder debugging: Native crashes and memory problems can be more difficult to diagnose than ordinary Java exceptions. (Oracle)
When to use	Use FFM when a Java application needs high-performance access to an existing native library, native operating-system API, native data structure, or off-heap memory .
When not to use	Do not use FFM merely because it is available. For functionality already provided efficiently by standard Java APIs, staying entirely within Java is generally simpler and safer. Also avoid FFM when the team has no need for native interoperability and cannot justify the additional native-code complexity.
Real-time use case — image processing	A Java application can call an existing C/C++ image-processing library without writing traditional JNI wrapper code. Java can pass native-memory data through MemorySegment and invoke native functions through Linker.
Real-time use case — machine learning	Java applications can interact with native ML/inference libraries where the high-performance implementation already exists in C/C++ or another native language.
Real-time use case — database/native drivers	A Java application can access native database or storage libraries when the required functionality is exposed through a native API.
Real-time use case — operating-system APIs	Applications can call native OS functions that are not conveniently exposed through standard Java APIs.
Real-time use case — high-performance networking	A specialized networking application can interact with native networking libraries or structures where direct native access is beneficial.
Real-time use case — memory-mapped files	MemorySegment can represent memory associated with mapped files, allowing structured access to data outside normal Java heap objects. (OpenJDK CVM)
Real-time use case — legacy native library	An organization with an existing C/C++ financial, scientific, encryption, or engineering library can integrate that library directly with Java instead of creating a JNI layer.
Example — calling native function	Conceptually: Java obtains a native function using SymbolLookup, describes its signature with FunctionDescriptor, and creates a

	MethodHandle using Linker. The resulting handle can then be invoked from Java. (OpenJDK CVM)
Example — native memory	MemorySegment represents the native memory and Arena controls its lifetime. This provides a structured model for allocating and accessing off-heap memory. (Oracle)
Alternate feature — JNI	Java Native Interface (JNI) is the traditional alternative when Java needs to communicate with native C/C++ code. It remains useful for existing JNI integrations and cases requiring mature, established JNI infrastructure.
Alternate feature — Java standard APIs	If the required functionality already exists in the JDK, a standard Java API is generally preferable because it avoids native-code complexity altogether.
Alternate feature — ByteBuffer	For simpler off-heap/binary-data scenarios, ByteBuffer.allocateDirect() can be sufficient when you don't need the broader native-function and memory capabilities of FFM.
Alternate feature — Unsafe	sun.misc.Unsafe has historically been used for low-level memory access, but it is not the preferred modern alternative . Java has been moving away from its memory-access methods, with FFM being one of the recommended standard APIs for such use cases. (Oracle Docs)
Best suited for	Java applications that need controlled, high-performance interoperability with native libraries or native/off-heap memory.
Not ideal for	Pure Java business applications, ordinary CRUD applications, typical web services, and applications where an equivalent standard Java API already exists.

FFM API vs Alternative Approaches

Requirement	FFM API	JNI	Direct ByteBuffer	Unsafe
Call native functions	✓ Excellent	✓ Excellent	✗	✗
Access native/off-heap memory	✓ Excellent	✓	✓ Good	✓

Pure Java API for integration	✓ Yes	✗ Requires native glue code	✓	✓
Need C/C++ JNI wrapper	✗ Generally no	✓ Usually	✗	✗
Memory lifetime management	✓ Arena	☐ Developer-managed/native	☐ GC-associated buffer object	✗ Very low-level
Safety	✓ Better structured safety	☐ Error-prone	✓ Relatively safe	✗ Low-level/unsafe
Native library integration	✓ Excellent	✓ Excellent	✗ Limited	✗
Existing legacy JNI code	☐ Migration required	✓ Best	✗	✗
Complex native structs	✓ Excellent	✓	☐ Limited	☐ Difficult
Platform-specific native API	✓	✓	✗	✗
Development complexity	Medium	High	Low	High
Recommended for new native integration	✓ Yes	☐ Usually prefer FFM	✗ Not for function calls	✗ Generally avoid
Long-term direction	✓ Modern standard API	Mature legacy/native integration	Useful for simpler buffer scenarios	Memory-access methods being deprecated for removal

Simple decision guide

Situation	Recommended choice
Need to call a C/C++ library from new Java code	FFM API
Existing large JNI integration	JNI , unless there is a reason to migrate

Need simple direct/off-heap byte storage	Direct ByteBuffer may be sufficient
Need structured native memory + native functions	FFM API
Need OS/native API access	FFM API
Pure Java application	Standard Java APIs
Considering Unsafe for new code	Prefer FFM or other supported Java APIs

Key takeaway: The Java 22 FFM API is essentially **Project Panama's production-ready native interoperability mechanism**. Its biggest advantage is that Java can interact with **native functions and foreign memory using a standard Java API**, avoiding much of the boilerplate and brittleness traditionally associated with JNI. ([Oracle Docs](#))

Aspect	Unnamed Variables & Patterns
Java Version	Java 22
Year	2024
Feature	Unnamed Variables & Patterns
UniqueClassName	UnnamedVariablesPatterns
JEP	JEP 456
Status	Final in Java 22
Purpose	Allows developers to use <code>_</code> for variables or pattern variables whose values are intentionally not needed . This makes code more concise and clearly communicates that a particular value is deliberately ignored.
Syntax	Examples: <code>catch (Exception _) { }</code> , <code>if (obj instanceof Point(int _, int y))</code> , and <code>for (var _ : collection)</code> .
Important point	<code>_</code> is not a normal variable that can be read. It explicitly means "I don't need this value."

Advantages and Disadvantages

Category	Details
Advantage 1 — Improved readability	Clearly communicates that a particular value is intentionally ignored.
Advantage 2 — Less meaningless naming	Avoids names such as ignored, unused, unusedValue, or exception.
Advantage 3 — Pattern matching becomes cleaner	Particularly useful when deconstructing records but only some components are required.
Advantage 4 — Avoids accidental use	Because <code>_</code> cannot be referenced as a variable, developers cannot accidentally use a value that was intended to be ignored.
Advantage 5 — Cleaner exception handling	Useful when the exception object itself is irrelevant.
Advantage 6 — Cleaner loops	Useful when a loop needs to execute a fixed number of times but doesn't need the loop variable.
Disadvantage 1 — Learning curve	Developers familiar with older Java versions may initially find <code>_</code> unusual.
Disadvantage 2 — <code>_</code> has special meaning	It cannot be used as an ordinary identifier for a variable.
Disadvantage 3 — Can hide useful information	Overusing <code>_</code> may conceal data that might actually be useful for debugging or future maintenance.
Disadvantage 4 — Requires Java 22+	Code using this feature cannot be compiled with older Java versions.
Disadvantage 5 — Migration consideration	Teams maintaining Java 8/11/17 applications cannot immediately use this syntax without upgrading their Java version.

When to Use

Situation	Use?	Reason
-----------	------	--------

Ignoring an exception object	✓ Yes	The exception may not be needed.
Ignoring one component of a record pattern	✓ Yes	Makes the intent explicit.
Loop variable is unnecessary	✓ Yes	Avoids meaningless variable names.
Only some pattern components are needed	✓ Yes	Makes pattern matching more readable.
Temporary/placeholder variable	□ Sometimes	Use _ only when the value is genuinely irrelevant.
Value may be needed for debugging	✗ No	Give it a meaningful name.
Value may be used later	✗ No	Don't discard potentially useful information.
Complex business logic	□ Carefully	Excessive unnamed variables can make logic harder to understand.
Supporting Java 8/11/17	✗ No	The syntax is unavailable.

Real-Time Use Cases

Use Case	Example	Why _ Helps
1. Ignoring an exception	java try { process(); } catch (IOException _) { useDefaultValue(); } 	The exception object isn't needed.
2. Record pattern — ignore one field	java record Point(int x, int y) {} if (obj instanceof Point(int _, int y)) { System.out.println(y); } 	x isn't required; only y matters.
3. Ignore multiple record components	java record Employee(String name, int age, String department) {} if (obj instanceof Employee(_, _, String dept)) { System.out.println(dept); } 	Clearly shows that only department is relevant.

4. Repeated operation	java for (int _ : IntStream.range(0, 10).toArray()) { performTask(); } 	The loop value itself isn't important.
5. Map entry pattern	java for (var entry : map.entrySet()) { // use entry } 	In pattern-based code, _ can explicitly represent components that aren't needed.
6. Switch pattern matching	java switch (obj) { case Point(int _, int y) -> System.out.println(y); default -> System.out.println("Other"); } 	Makes ignored pattern components explicit.
7. Lambda/pattern scenarios	When a supported pattern introduces a value that isn't needed, _ communicates that the value is deliberately discarded.	Avoids artificial names such as ignoredValue.

Example: Before vs Java 22

Before Java 22	Java 22
catch (IOException ignored) { ... }	catch (IOException _) { ... }
Point(int x, int y) when x isn't used	Point(int _, int y)
case Point(int x, int y) -> use(y);	case Point(int _, int y) -> use(y);
Meaning of variable must be inferred from name	_ explicitly communicates ignored value

Alternate Features

Alternative	Description	When preferable
Named variables	Give every value a meaningful variable name.	When the value may be useful or its identity improves readability.
var	Local variable type inference can reduce verbosity, but the variable still has a name.	When the value is needed.

Traditional pattern variables	Use named pattern variables such as <code>Point(int x, int y)</code> .	When multiple components need to be used.
Record patterns	Destructure records directly.	When you need several record components; combine with <code>_</code> when some components are irrelevant.
switch pattern matching	Match types/records without manual casts.	When control flow depends on the object's structure.
Helper methods	Move unused-value handling into a method whose parameters express what matters.	When the ignored information is part of a larger reusable operation.

Unnamed Variables & Patterns vs Alternatives

Requirement	Unnamed Variables & Patterns	Named Variables	Record Patterns	Var
Explicitly ignore a value	✓ Excellent	☐ Naming workaround	✓ With <code>_</code>	✗
Pattern matching	✓ Excellent	✓	✓ Excellent	✗
Record destructuring	✓ Excellent	✓	✓ Excellent	✗
Improve readability when value is unused	✓ Excellent	☐	✓	☐
Use the value later	✗	✓ Best	✓	✓
Java 22+ required	✓	✗	Depends on feature	✗
Prevent accidental use	✓ Yes	✗	Partly	✗
Simple	☐	✓ Best	✗	✓

ordinary local variable				
-------------------------	--	--	--	--

Best-Practice Decision Table

Scenario	Recommendation
Value is definitely irrelevant	✓ Use _
Only one field of a record matters	✓ Use unnamed pattern for the other fields
Exception details are irrelevant	✓ catch (Exception _)
Loop value is irrelevant	✓ Use an unnamed variable where applicable
Value could be useful during troubleshooting	✗ Keep a named variable
Business meaning is important	✗ Use a meaningful name
Multiple developers unfamiliar with Java 22	□ Introduce the feature gradually
Maintaining old Java versions	✗ Don't use it

Key takeaway: Unnamed Variables & Patterns is mainly a **readability and intent-expression feature**. It tells both the compiler and other developers: **“This value exists, but I intentionally don't care about it.”**

The biggest practical benefit appears with **pattern matching and record patterns**, where you may need only one or two components of a larger object.

Aspect	Class-File API (Preview)
Java Version	Java 22
Year	2024
Feature	Class-File API (Preview)
UniqueClassName	ClassFileAPI

JEP	JEP 457
Status	Preview in Java 22
Package	java.lang.classfile
Purpose	Provides a standard Java API for reading, parsing, transforming, and generating JVM class files .
Problem it solves	Before this API, developers commonly relied on third-party bytecode libraries or internal JDK APIs to inspect and manipulate .class files.
Core idea	Treat a JVM class file as structured data that Java code can inspect and transform through a supported API.
Main operations	Read existing class files, inspect class-file structures, transform bytecode/class-file elements, and generate new class files.
Important classes/concepts	ClassFile, ClassModel, MethodModel, FieldModel, CodeModel, ClassTransform, and related class-file elements.
Advantages	1. Standard JDK API: Reduces dependence on non-standard/internal APIs. 2. No external library required: Bytecode processing can be performed using the JDK API itself. 3. Class-file generation: Can programmatically create valid JVM class files. 4. Bytecode transformation: Useful for modifying existing classes. 5. Better maintainability: A standard API is preferable to depending on JDK internals. 6. Designed for JVM evolution: Intended to evolve alongside Java class-file format changes. 7. Useful for tooling: Particularly valuable for frameworks, compilers, instrumentation tools, and build tools.
Disadvantages	1. Preview feature: In Java 22 it required preview support and could change before becoming final. 2. Advanced API: Requires knowledge of JVM class files and bytecode concepts. 3. Debugging complexity: Incorrect bytecode transformations can produce difficult-to-diagnose runtime problems. 4. Not needed for normal applications: Most business applications never need to manipulate .class files directly. 5. Version compatibility: Generated/transformed class files must remain compatible with the target JVM version. 6. Migration effort: Existing projects using ASM or other

	bytecode libraries may require significant redesign to move to the Class-File API.
When to use	Use it when developing bytecode tools, Java agents, instrumentation frameworks, compilers, code generators, class analyzers, or JVM-oriented development tools.
When not to use	Don't use it for ordinary application development, CRUD applications, simple reflection-based tasks, or situations where you only need to inspect normal object metadata.
Real-time use case — APM agent	An application-performance-monitoring agent can transform class files to add timing, tracing, or metrics around methods.
Real-time use case — logging/instrumentation	A framework can inject logging instructions into selected methods without requiring developers to modify the original source code.
Real-time use case — code coverage	A coverage tool can transform methods to insert counters that record which instructions or branches were executed.
Real-time use case — security scanning	A static analysis tool can inspect class-file structures to detect suspicious bytecode patterns or unsupported constructs.
Real-time use case — framework proxy generation	A framework can dynamically generate specialized classes instead of relying entirely on reflection.
Real-time use case — compiler/code generator	A compiler or DSL implementation can generate JVM .class files directly from another language or domain-specific representation.
Real-time use case — build tooling	A build tool can inspect or transform class files during compilation/package processing.
Example — reading a class file	Conceptually, a tool can obtain a ClassModel from a .class file and iterate over its fields, methods, attributes, and bytecode instructions.
Example — transformation	A framework can apply a ClassTransform to a class and modify selected class-file elements while leaving the rest unchanged.
Example — generation	A code-generation tool can use ClassFile to construct a new class containing fields, methods, and JVM bytecode, then write the resulting bytes to a .class file.

Simplified Example

A conceptual transformation can look like:

```

ClassFile cf = ClassFile.of();

ClassTransform transform = ClassTransform
    .endHandler(classBuilder ->
        System.out.println("Class processed"));

byte[] transformed = cf.transformClass(
    originalBytes,
    transform
);

```

The important idea is that the API provides **structured access to class-file contents** rather than requiring developers to manually parse the binary .class format.

Alternate Features / Technologies

Alternative	Purpose	When preferable
ASM	Popular third-party bytecode manipulation library	Excellent choice when you need mature bytecode transformation today, especially for existing projects.
Byte Buddy	High-level runtime code generation and instrumentation	Prefer when you want to generate/proxy/instrument Java classes without working directly with low-level JVM bytecode details.
Javassist	High-level Java bytecode manipulation	Useful when source-like manipulation is more convenient than direct bytecode APIs.
Java Reflection API	Inspects classes, methods, fields, annotations, etc. at runtime	Best when you only need metadata or to invoke existing methods; no bytecode modification is required.
java.lang.instrument	Java-agent instrumentation framework	Best when your main requirement is transforming classes during JVM startup/runtime.
MethodHandle / invokedynamic	Dynamic method invocation and linking	Prefer when the problem is dynamic invocation rather than creating or modifying complete class files.

Class-File API vs Alternatives

Requirement	Class-File API	ASM	Byte Buddy	Reflection	Instrumentation API
Read .class structure	✓ Excellent	✓ Excellent	✓	☐ Runtime metadata	☐
Generate .class files	✓ Yes	✓ Yes	✓ Yes	✗	✗
Transform bytecode	✓ Yes	✓ Excellent	✓ Excellent	✗	✓
Standard JDK API	✓ Yes	✗	✗	✓	✓
High-level API	☐ Medium/low	☐ Low-level	✓ High-level	✓	Medium
No external dependency	✓	✗	✗	✓	✓
Runtime Java-agent work	✓ Possible	✓ Excellent	✓ Excellent	✗	✓ Best suited
Simple metadata inspection	☐ Overkill	✗	✗	✓ Best	✗
Fine-grained bytecode control	✓ Yes	✓ Excellent	✓	✗	Depends on transformer
Beginner friendly	?	✗	✓	✓	?
Java 22 preview	✓	✗	✗	✗	✗

When to Choose Which

Scenario	Recommended
Need to manipulate JVM class-file	Class-File API

structures using only the JDK	
Building a new bytecode framework around standard Java APIs	Class-File API
Mature production bytecode manipulation today	ASM
High-level runtime class generation	Byte Buddy
Simple runtime class inspection	Reflection
Java agent / runtime instrumentation	java.lang.instrument + ASM/Byte Buddy/Class-File API
Generating classes from another language/compiler	Class-File API or ASM
Ordinary Spring/REST/CRUD application	Don't use Class-File API

Key Takeaway

Class-File API (Preview) in Java 22 is aimed primarily at **framework and tooling developers**, not everyday application developers.

Its biggest advantage is providing a **standard JDK API for class-file parsing, transformation, and generation**, reducing reliance on JDK-internal APIs and external bytecode libraries for certain use cases.

For a normal business application, however, **Reflection, Method Handles, or ordinary Java APIs are usually much more appropriate**. For sophisticated bytecode manipulation, **ASM or Byte Buddy** remained important alternatives, particularly for mature production ecosystems.

Aspect	Launch Multi-File Source-Code Programs
Java Version	Java 22
Year	2024
Feature	Launch Multi-File Source-Code Programs
UniqueClassName	MultiFileSourceCode
JEP	JEP 458

Status	Final in Java 22
Purpose	Extends Java's source-code launcher so a Java program can be launched directly from source files even when the program consists of multiple source files .
Before Java 22	java MyProgram.java worked conveniently for a single source file, but launching a program consisting of multiple source files directly from source was more restrictive.
Java 22 improvement	The java launcher can locate and compile additional source files needed by the initial source file, allowing a small multi-file application to be launched without explicitly compiling everything first.
Main benefit	Provides a convenient compile-and-run workflow for small multi-file Java programs while retaining the traditional Java compilation model for production applications.

Advantages and Disadvantages

Category	Details
Advantage 1 — Easy execution	Developers can run a multi-file Java application directly from source without manually invoking javac first.
Advantage 2 — Faster experimentation	Very useful for prototypes, demonstrations, experiments, and learning exercises.
Advantage 3 — Less build boilerplate	A small program may not need a Maven/Gradle project just to run several related source files.
Advantage 4 — Natural Java development	Developers can split code into multiple classes while still using the simple source-launching experience.
Advantage 5 — Convenient scripts/tools	Small command-line utilities can be distributed as source files and launched with minimal setup.

Advantage 6 — Incremental adoption	A developer can start with source launching and later move the project to a conventional build system when it grows.
Disadvantage 1 — Not a replacement for a build system	Large applications still benefit from Maven, Gradle, or another build tool for dependency management, testing, packaging, and reproducible builds.
Disadvantage 2 — Compilation still occurs	The launcher must compile the necessary source files before executing the program.
Disadvantage 3 — Dependency management limitations	External dependencies and complex project configurations are better handled by build tools.
Disadvantage 4 — Less suitable for production deployment	Production systems generally benefit from precompiled artifacts such as JARs or container images.
Disadvantage 5 — Potentially slower startup	Source launching can involve source discovery and compilation, making it less convenient for repeated production execution.
Disadvantage 6 — Can encourage oversized scripts	If a project continually grows while remaining source-launched, its structure and build process can become harder to manage.

Basic Example

Suppose we have three source files:

Main.java
Customer.java
CustomerService.java

Main.java:

```
public class Main {
    public static void main(String[] args) {
        Customer customer = new Customer("Anurag");

        CustomerService service = new CustomerService();
        service.printCustomer(customer);
    }
}
```

Customer.java:

```

public class Customer {
    private final String name;

    public Customer(String name) {
        this.name = name;
    }

    public String name() {
        return name;
    }
}

```

CustomerService.java:

```

public class CustomerService {
    public void printCustomer(Customer customer) {
        System.out.println("Customer: " + customer.name());
    }
}

```

With Java 22, the source launcher can resolve the additional source files required by the initial source file, providing a simpler development experience for this kind of small application.

When to Use

Scenario	Use?	Reason
Small command-line utility	✓ Yes	Minimal setup and easy execution
Learning Java	✓ Yes	Lets learners split examples into multiple classes
Coding demonstration	✓ Yes	Convenient for demos
Proof of concept	✓ Yes	Quickly test an idea
Small internal tool	✓ Yes	Avoids unnecessary build configuration
Temporary automation script	✓ Yes	Easy source-based execution

Large enterprise application	✗ No	Maven/Gradle and structured builds are more appropriate
Complex dependency graph	✗ No	Build tools provide much better dependency management
Production application	ⓧ Usually no	Prefer compiled and packaged artifacts
CI/CD production pipeline	ⓧ Usually no	Reproducible builds and artifacts are generally preferable
Multi-module project	✗ No	Use Maven/Gradle or another build system

Real-Time Use Cases

Use Case	Example	Benefit
1. Developer utility	Main.java uses FileProcessor.java and Report.java	Multiple classes can be launched directly from source.
2. Coding exercise	Main.java, Student.java, and GradeCalculator.java	Students can learn class organization without first learning a build tool.
3. Data-processing script	Main.java + CsvReader.java + Statistics.java	A small data utility can remain source-based.
4. Internal administration tool	Main.java + Database.java + ReportGenerator.java	Convenient for small one-off utilities.
5. Proof of concept	Main.java + several supporting classes	Developers can quickly test architecture or an algorithm.
6. Technical demonstration	A conference/tutorial example split across several classes	Easier to demonstrate real application structure than putting everything into one file.
7. Automation tool	Main.java + FileCleaner.java + Config.java	Useful for lightweight operational scripts.
8. Prototyping a library	A small launcher plus several model/helper	Developers can experiment before introducing a complete build

	classes	project.
--	---------	----------

Alternate Features

Alternative	Purpose	When Preferable
java MyProgram.java source launcher	Run a single source file directly	Best for genuinely small one-file programs.
javac + java	Explicitly compile and execute Java classes	Best when you want traditional, explicit compilation control.
Maven	Build, dependency management, testing, packaging	Best for structured enterprise and library projects.
Gradle	Build automation, dependency management, testing, packaging	Best for complex or highly customizable builds.
JShell	Interactive Java experimentation	Best when you don't need a multi-class source program.
Executable JAR	Package compiled application into a distributable artifact	Best for production deployment.

Multi-File Source Launcher vs Alternatives

Requirement	Multi-File Source-Code Launcher	Single-File Source Launcher	javac + java	Maven/Gradle
Run without explicit compilation command	✓ Yes	✓ Yes	✗	✗
Multiple source files	✓ Yes	⌚ Limited	✓	✓
Quick experimentation	✓ Excellent	✓ Excellent	✓	⌚
Minimal setup	✓ Excellent	✓ Excellent	✓	✗
External dependency	⌚ Limited	⌚ Limited	⌚	✓ Excellent

management			Manual	
Testing lifecycle	✗	✗	? Manual	✓ Excellent
Packaging	✗	✗	? Manual	✓ Excellent
Production deployment	? Usually not	? Usually not	✓	✓ Best
Large projects	✗	✗	?	✓ Best
Learning/demo programs	✓ Excellent	✓ Excellent	✓	?
CI/CD build management	?	?	✓	✓ Best

Decision Guide

Situation	Recommended Approach
One tiny Java program	Single-file source launcher
Small program with several .java files	Multi-file source-code launcher
Need explicit compilation	javac + java
Need dependency management	Maven/Gradle
Need automated testing and packaging	Maven/Gradle
Production deployment	Build/package into JAR or another deployable artifact
Interactive experimentation	JShell

Key Takeaway

Launch Multi-File Source-Code Programs makes Java's source-file launcher much more practical for **small programs that naturally consist of multiple classes**.

Its sweet spot is:

“I have a small Java program with several source files, but I don't want to create a complete Maven/Gradle build just to run it.”

For **production and large enterprise applications**, however, this feature should not replace a proper build system. **Maven or Gradle** remains the better choice when dependency management, testing, packaging, reproducibility, and CI/CD become important.

Aspect	String Templates (Second Preview)
Java Version	Java 22
Year	2024
Feature	String Templates (Second Preview)
UniqueClassName	StringTemplates
JEP	JEP 459
Status	Second Preview in Java 22
Purpose	Provides a more readable and structured way to combine literal text with embedded expressions , while also allowing template processors to control how the resulting value is created.
Basic idea	Instead of manually concatenating strings, developers can embed expressions directly into a template.
Important status note	String Templates were previewed in Java 21 and again in Java 22, but the feature was not finalized and was subsequently withdrawn from the JDK roadmap. Therefore, it should not be treated as a normal Java 22 production feature .
Basic syntax in Java 22 preview	<pre>java
String name = "Anurag";
String message = STR."Hello, \{name}!";
</pre>
Expression embedding	Expressions can be embedded using <code>\{...}</code> inside the template.
Built-in processor	STR was the built-in string-template processor for ordinary string interpolation.
Other processors	The design also allowed custom processors, making templates useful for things beyond ordinary string concatenation.

Advantages and Disadvantages

Category	Details
Advantage 1 — Better readability	Embedded expressions are easier to read than long chains of + concatenation.
Advantage 2 — Less boilerplate	Reduces repetitive string concatenation code.
Advantage 3 — Expression embedding	Java expressions can be placed directly inside the template.
Advantage 4 — Type-aware processing	Unlike simple textual interpolation, template processors can receive structured template fragments and expression values before producing the final result.
Advantage 5 — Custom processors	Developers can theoretically create processors for specialized purposes such as SQL, JSON, HTML, logging, or validation.
Advantage 6 — Potentially safer APIs	Custom processors can validate or transform embedded values instead of blindly concatenating text.
Advantage 7 — Improved maintainability	Large formatted strings can become easier to understand and modify.
Disadvantage 1 — Preview feature	In Java 22 it required preview support and was not a finalized language feature.
Disadvantage 2 — Learning curve	Developers needed to understand template processors in addition to ordinary string syntax.
Disadvantage 3 — Syntax uncertainty	Because the feature was previewed and ultimately withdrawn, code written against it carries migration risk.
Disadvantage 4 — Not necessary for simple strings	Ordinary string concatenation is perfectly adequate for many simple cases.
Disadvantage 5 — Custom processors add complexity	A custom processor can make code harder to understand if the processing rules aren't obvious.
Disadvantage 6 — Ecosystem compatibility	Code using the Java 22 preview syntax cannot simply be compiled as ordinary Java source on versions that don't support that preview.
Disadvantage 7 —	String templates themselves do not automatically make SQL ,

Security is not automatic	HTML, shell commands, etc. safe. Safety depends on the processor/API being used correctly.
----------------------------------	---

When to Use

Situation	Use?	Reason
Simple string interpolation	✓ Conceptually useful	Cleaner than repeated + operations.
Constructing readable messages	✓	Embedded expressions make the intent clear.
Generating structured text	✓	A custom processor can provide additional processing.
Domain-specific templating	✓	Custom processors can implement domain-specific behavior.
SQL generation	⚠ Only with a properly designed processor/API	Never assume interpolation alone prevents SQL injection.
HTML generation	⚠ Only with proper escaping	Values must be HTML-escaped appropriately.
Large production application in Java 22	✗ Generally avoid	It was still a preview feature.
Long-term library API	✗	Preview features create compatibility and migration risks.
Simple concatenation of 2-3 values	⚠	+ may be simpler and clearer.
Applications requiring Java 17/21 compatibility	✗	The Java 22 preview syntax isn't available there.

Real-Time Use Cases

Use Case	Example	Benefit
1. User-facing message	java String name = "Anurag"; String msg =	More readable than "Welcome, " + name +

	STR."Welcome, \{name}!"; 	"!".
2. Logging	java String user = "admin"; int count = 15; String log = STR."User=\{user}, count=\{count}"; 	Makes structured log-message construction concise.
3. Report generation	java String product = "Laptop"; double price = 75000; String report = STR."Product: \{product}, Price: ₹\{price}"; 	Useful for readable formatted output.
4. Configuration messages	java String host = "localhost"; int port = 8080; String endpoint = STR."http://\{host}:\{port}"; 	Cleaner construction of dynamic configuration strings.
5. JSON-like text	A template can combine fixed JSON structure with dynamic values.	Makes generated text easier to read, although a JSON library is usually safer for real JSON serialization.
6. SQL-related processing	A custom processor could distinguish literal SQL fragments from embedded values.	Potentially allows validation/parameterization rather than naïve concatenation.
7. HTML generation	A custom processor could HTML-escape embedded values.	Can provide safer structured output when correctly designed.
8. Domain-specific DSL	A processor can interpret template fragments according to application-specific rules.	Extends templates beyond simple string concatenation.

Example: Traditional Java vs String Templates

Approach	Example	Readability
String concatenation	String msg = "Hello " + name + ", your balance is " + balance;	Ⓜ Moderate
String.format()	String msg =	✓ Good

	<code>String.format("Hello %s, your balance is %.2f", name, balance);</code>	
MessageFormat	<code>MessageFormat.format("H ello {0}, your balance is {1}", name, balance);</code>	✓ Good
String Template	<code>String msg = STR."Hello \{name}, your balance is \{balance}";</code>	✓ Very readable

Custom Processor Concept

One of the more powerful aspects of the proposal was that templates were not limited to simply producing a String.

Conceptually:

```
String name = "Anurag";
```

```
String result = STR."Hello, \{name}!";
```

uses the STR processor.

A custom processor could instead conceptually do something like:

```
Template
↓
Template Processor
↓
Validation / escaping / transformation
↓
Result
```

This was important because the proposal was designed to make **template processing extensible**, rather than merely introducing syntactic sugar for +.

Alternate Features

Alternative	Purpose	When to Prefer
String concatenation (+)	Basic string construction	Best for simple, short expressions
String.format()	Formatted string	Good when formatting numbers,

	generation	dates, widths, or precision
formatted()	Formatting an existing format string	Useful for familiar printf-style formatting
MessageFormat	Parameterized messages	Useful for localization and structured messages
Text Blocks	Multi-line string literals	Best when the main problem is multi-line text rather than interpolation
StringBuilder	Efficient incremental string construction	Good for loops or complex programmatic text generation
Dedicated serialization libraries	Generate JSON/XML/etc. correctly	Preferable to manually constructing structured formats
JDBC PreparedStatement	Parameterized SQL	Prefer this over constructing SQL with string interpolation

String Templates vs Alternatives

Requirement	String Templates	+ Concatenation	String.format()	Message Format	Text Blocks
Embedded expressions	✓ Excellent	✓	✓	✓	✗
Simple concatenation	✓	✓ Best	?	?	✗
Numeric formatting	? Processor-dependent	✗	✓ Excellent	✓	✗
Multi-line text	✓	?	?	?	✓ Best
Custom processing	✓ Excellent	✗	✗	?	✗
Localization	? Processor-dependent	✗	?	✓ Excellent	✗

No preview dependency	✗ in Java 22	✓	✓	✓	✓
Simple to learn	?	✓ Best	✓	?	✓
Structured output safety	? Depends on processor	✗	✗	✗	✗
Long-term Java compatibility	✗	✓	✓	✓	✓

What to Use Instead in Practice

Requirement	Recommended Choice
"Hello " + name	+ concatenation
"Name: %s, Age: %d" formatting	String.format() / formatted()
Multi-line SQL/JSON/HTML literal	Text blocks + appropriate library/API
JSON generation	Jackson/Gson/JSON-B, rather than manually concatenating JSON
SQL with user input	PreparedStatement, not string interpolation
Localized application messages	ResourceBundle / MessageFormat
Efficient repeated string construction	StringBuilder
Java 22 experimentation with template syntax	String Templates preview, with the understanding that it was not finalized

Final Assessment

Question	Answer
Was it introduced in Java 22?	✓ Yes

Was it final in Java 22?	✗ No
Was it the second preview?	✓ Yes — JEP 459
Was it useful for interpolation?	✓ Yes
Was it more powerful than simple interpolation?	✓ Yes, because of template processors
Should you use it for a Java 22 production project?	🚫 Generally no , because it was still a preview feature
Should you learn the concept?	✓ Yes, especially for understanding Java's exploration of extensible template processing
Current practical alternative	+ , String.format() , formatted() , MessageFormat , Text Blocks , or dedicated libraries , depending on the problem

Key takeaway: Java 22's **String Templates (Second Preview)** offered elegant interpolation such as `STR."Hello, \{name}"` and, more importantly, an extensible **template-processor model**. However, it remained a preview feature and was ultimately **withdrawn rather than becoming a permanent Java language feature**. For production Java development, use the appropriate established alternative instead.

Aspect	Vector API (Seventh Incubator)
Java Version	Java 22
Year	2024
Feature	Vector API (Seventh Incubator)
UniqueClassName	VectorAPI
JEP	JEP 460
Status	Seventh Incubator in Java 22
Purpose	Provides an API for expressing vector/SIMD computations so the JVM can compile them into efficient hardware vector instructions when supported by the processor.

SIMD meaning	Single Instruction, Multiple Data — one CPU instruction can process multiple values simultaneously.
Main goal	Improve performance of data-parallel computations while allowing developers to write the algorithm in portable Java rather than architecture-specific intrinsic code.
Important point	The Vector API is primarily a performance-oriented API . It does not automatically make every Java calculation faster. The workload must contain suitable data-parallel operations.
Core API	Classes such as VectorSpecies, IntVector, FloatVector, DoubleVector, ByteVector, and related vector types.
Typical operations	Vector addition, subtraction, multiplication, comparisons, masks, reductions, lane-wise operations, and loads/stores from arrays or memory segments.

Advantages and Disadvantages

Category	Details
Advantage 1 — SIMD performance	Can process multiple array elements simultaneously using CPU vector instructions.
Advantage 2 — Portable Java code	Developers can express vector algorithms in Java instead of writing separate architecture-specific implementations.
Advantage 3 — JVM optimization	The JIT compiler can map Vector API operations to appropriate hardware instructions available on the target CPU.
Advantage 4 — Excellent for data-parallel workloads	Particularly effective for operations applied independently to large collections of primitive values.
Advantage 5 — Supports multiple primitive types	APIs are available for integer and floating-point vector computations, among others.
Advantage 6 — Better performance without native code	Can provide SIMD-style performance while remaining within the Java ecosystem.
Advantage 7 — Useful for performance-critical libraries	Numerical, scientific, image-processing, cryptographic-support, and machine-learning algorithms can benefit when their operations

	map well to SIMD.
Disadvantage 1 — Incubator feature	In Java 22 it was still an incubating API, so its design could change in future releases.
Disadvantage 2 — More complex code	Vectorized code is generally more complicated than a straightforward scalar loop.
Disadvantage 3 — Not universally faster	Small datasets, branch-heavy algorithms, irregular memory access, or already highly optimized code may see little benefit.
Disadvantage 4 — Hardware dependent for performance	The actual performance depends heavily on CPU vector capabilities and the JVM's ability to generate appropriate instructions.
Disadvantage 5 — Debugging complexity	Lane-wise operations, masks, species, and vector boundaries can make debugging less intuitive.
Disadvantage 6 — Tail handling	Algorithms whose data size isn't a multiple of the vector length need appropriate handling for remaining elements.
Disadvantage 7 — Benchmarking is essential	Performance assumptions should be validated with realistic benchmarks rather than assuming vectorization automatically improves performance.

Simple Scalar vs Vector Example

Approach	Example	Concept
Traditional scalar loop	<code>for (int i = 0; i < a.length; i++) result[i] = a[i] + b[i];</code>	Processes one pair of values at a time.
Vector API	<code>java
var species =
IntVector.SPECIES_PREFERRED;

for (int i
= 0; i < a.length; i += species.length()) {
var va
= IntVector.fromArray(species, a, i);
var vb =
IntVector.fromArray(species, b, i);
va.add(vb).intoArray(result, i);
}
</code>	Processes multiple integer elements per vector operation.

The JVM can potentially translate the vector operations into hardware SIMD instructions.

When to Use

Situation	Use?	Reason
Large numeric arrays	✓ Excellent	Many independent operations can be processed in parallel.
Image pixel processing	✓ Excellent	Pixel/channel operations are often naturally data-parallel.
Matrix/vector calculations	✓ Excellent	Many mathematical operations map well to SIMD.
Signal processing	✓ Excellent	Filtering and transformations often process repeated numeric values.
Scientific computing	✓ Excellent	Large floating-point datasets can benefit from vectorization.
Machine-learning primitives	✓ Good	Some low-level numerical kernels are highly vectorizable.
Financial numerical calculations	✓ Good	Large batches of independent calculations can benefit.
Simple CRUD application	✗ No	Database/network/application logic isn't usually SIMD-bound.
Branch-heavy business logic	✗ Usually no	SIMD provides limited benefit when operations diverge heavily.
Tiny arrays	✗ Usually no	Vectorization overhead may outweigh the benefit.
Irregular pointer/data structures	✗ Usually no	SIMD works best with predictable, contiguous data.
Already optimized native library	? Benchmark first	Native SIMD implementation may already be superior.

Real-Time Use Cases

Use Case	Example	Why Vector API Helps
1. Image processing	Brightness adjustment: <code>pixel[i] = pixel[i] * factor</code>	Thousands/millions of pixels can be processed in vector-sized batches.
2. Audio processing	Apply gain to a large PCM audio buffer	Independent samples can be multiplied in parallel.
3. Signal processing	FIR filtering, normalization, signal transformations	Repeated numerical operations map naturally to SIMD.
4. Machine learning	Vector/matrix arithmetic in a numerical kernel	Multiple numerical values can be processed simultaneously.
5. Financial analytics	Calculate returns or risk metrics for millions of values	Batch numerical operations are highly data parallel.
6. Scientific computing	Numerical simulations and array-based calculations	Large homogeneous datasets can benefit from SIMD.
7. Compression	Process blocks of bytes/integers during encoding/decoding	Block-oriented operations may be vectorizable.
8. Cryptographic support	Certain low-level transformations involving independent lanes	SIMD can accelerate suitable numerical/bitwise operations, although specialized crypto implementations may still be preferable.
9. Computer vision	Pixel/channel comparisons and transformations	Many pixels can undergo identical operations simultaneously.
10. Game engines	Transform or update large batches of numeric game data	Position, velocity, or other numeric arrays can sometimes be processed in parallel.

Example — Image Brightness

A scalar implementation:

```
for (int i = 0; i < pixels.length; i++) {
    pixels[i] = pixels[i] + brightness;
}
```

A Vector API implementation can process several pixels per iteration:

```
VectorSpecies<Integer> species = IntVector.SPECIES_PREFERRED;
```

```
for (int i = 0; i < pixels.length; i += species.length()) {  
    IntVector v = IntVector.fromArray(species, pixels, i);  
    v.add(brightness).intoArray(pixels, i);  
}
```

The exact performance depends on the JVM, CPU, data size, memory access pattern, and surrounding code.

When NOT to Use It

Situation	Why Avoid
Business rules	Conditional business logic generally doesn't benefit from SIMD.
Database access	Network/database latency dominates rather than arithmetic throughput.
REST API controllers	Vectorization usually provides little value.
Small calculations	Setup and processing overhead can outweigh gains.
Highly irregular algorithms	SIMD works best when operations are uniform across data.
Frequent branches	Divergent operations reduce the benefit of vectorization.
Maintainability is the priority	A simple scalar implementation may be preferable if performance isn't a bottleneck.
No benchmark evidence	Don't introduce complexity without demonstrating a meaningful performance improvement.

Alternate Features / Technologies

Alternative	Purpose	When Preferable
Traditional Java loops	Scalar processing of	Best default when performance is

	arrays/collections	adequate and simplicity matters.
JIT auto-vectorization	JVM may automatically convert suitable scalar loops into SIMD instructions	Prefer first when a simple loop already receives sufficient optimization.
Parallel Streams	Parallelize independent operations across CPU cores	Useful when the workload benefits from thread-level parallelism rather than SIMD alone.
ForkJoinPool	Explicit CPU-parallel task execution	Better when work can be divided into independent tasks.
FFM API	Access native functions and foreign memory	Prefer when using an existing native SIMD/numerical library rather than implementing the kernel in Java.
JNI	Traditional Java/native interoperability	Useful for existing native libraries, although FFM is the more modern native-interoperability API.
Native SIMD libraries	Architecture-specific optimized implementations	Prefer when maximum hardware-specific performance is required and native dependencies are acceptable.

Vector API vs Alternatives

Requirement	Vector API	Scalar Java	Auto-vectorization	Parallel Streams	Native SIMD
SIMD/vector operations	✓ Excellent	✗	✓ Sometimes	✗	✓ Excellent
Explicit vector control	✓ Yes	✗	✗	✗	✓
Pure Java	✓ Yes	✓	✓	✓	✗
Easy to write	⌚ Medium	✓ Best	✓ Best	✓	
Multiple CPU cores	✗ Not primarily	✗	✗	✓ Yes	Depends
Data-parallel	✓	✓	✓	✓	

arithmetic	Excellent				
Hardware-specific tuning	? JVM controlled	?	?	✗	✓ Best
External native dependency	✗	✗	✗	✗	✓
Maintainability	?	✓ Excellent	✓ Excellent	✓	
Maximum low-level control	✓	✗	✗	✗	✓ Best
Suitable for ordinary applications	? Rarely necessary	✓ Yes	✓ Yes	?	✗
Java 22 status	Seventh Incubator	Stable	JVM capability	Stable	External technology

Decision Guide

Requirement	Recommended Approach
Simple array calculation	Normal Java loop
Simple loop already optimized by JIT	Let the JVM auto-vectorize
Need explicit SIMD operations in Java	Vector API
Huge numeric dataset with independent operations	Vector API + benchmarking
Need to use all CPU cores	Parallelism + possibly Vector API
Existing optimized C/C++ numerical library	FFM API / native integration
Need maximum architecture-specific performance	Native SIMD library
Typical web/enterprise application	Normal Java code

Key Takeaway

Vector API (Seventh Incubator) in Java 22 is designed for **high-performance, data-parallel computation**. Its key advantage is allowing Java developers to explicitly express SIMD operations while letting the JVM map those operations to suitable CPU vector instructions.

The important distinction is:

Vector API = explicit data-level parallelism (SIMD)

Parallel Streams/ForkJoin = task/thread-level parallelism

For ordinary Java applications, start with **simple scalar code and let the JIT optimize it**. Introduce the Vector API only when profiling shows a genuine **CPU-bound, data-parallel bottleneck**, and confirm the improvement with proper benchmarks.

Aspect	Stream Gatherers (Preview)
Java Version	Java 22
Year	2024
Feature	Stream Gatherers (Preview)
UniqueClassName	StreamGatherers
JEP	JEP 461
Status	Preview in Java 22
Purpose	Provides a mechanism for defining custom intermediate stream operations that cannot be expressed conveniently using the standard Stream API.
Main problem solved	Standard operations such as map(), filter(), and flatMap() cover many cases, but more sophisticated stateful operations can be difficult or inefficient to express. Gatherers allow developers to define reusable intermediate operations.
Key API	java.util.stream.Gatherer
Basic concept	A gatherer receives stream elements, maintains optional state, and can emit zero, one, or multiple output elements.
Typical use cases	Windowing, batching, incremental aggregation, custom transformations, stateful processing, and operations that depend on previously seen elements.

Advantages and Disadvantages

Category	Details
Advantage 1 — Custom intermediate operations	Allows developers to create stream operations beyond the built-in map, filter, flatMap, distinct, etc.
Advantage 2 — Reusable logic	A complex stream operation can be packaged as a reusable Gatherer instead of duplicated across pipelines.
Advantage 3 — Stateful processing	Gatherers can maintain state while processing elements, making sliding windows and running calculations easier to implement.
Advantage 4 — Composable	A gatherer participates in a stream pipeline and can be combined with existing stream operations.
Advantage 5 — Potentially efficient	Custom operations can process elements incrementally instead of first materializing the entire stream into an intermediate collection.
Advantage 6 — Supports parallel processing design	The API includes concepts for integrating custom gathering operations with stream execution and parallelism.
Advantage 7 — Better abstraction	Domain-specific stream transformations can be expressed as named reusable operations.
Disadvantage 1 — Preview feature	In Java 22, Stream Gatherers was a preview feature and required preview support.
Disadvantage 2 — Higher complexity	Implementing a gatherer is significantly more complicated than using ordinary map(), filter(), or reduce().
Disadvantage 3 — Harder debugging	Stateful stream behavior can be harder to debug than straightforward imperative code.
Disadvantage 4 — Misuse can reduce readability	A complicated custom gatherer can make a pipeline harder for developers to understand.
Disadvantage 5 — Parallel semantics require care	A gatherer must correctly handle state and combination behavior if it is intended to work with parallel streams.

Disadvantage 6 — Not needed for simple operations	Most everyday stream transformations can already be expressed cleanly with standard Stream methods.
Disadvantage 7 — Preview compatibility	Applications using the Java 22 preview API may require changes if the API evolves between releases.

Basic Example — Fixed-Size Windowing

Suppose you have:

1, 2, 3, 4, 5, 6

and want:

[1, 2, 3]

[4, 5, 6]

A gatherer can encapsulate the stateful batching behavior so that the stream pipeline remains composable.

Conceptually:

```
var result = numbers.stream()
    .gather(window(3))
    .toList();
```

The important idea is that `window(3)` becomes a **custom intermediate stream operation**.

When to Use

Situation	Use?	Reason
Custom stateful stream operation	✓ Excellent	This is the primary purpose of Gatherers.
Sliding windows	✓ Excellent	Naturally requires state across elements.
Fixed-size batching	✓ Excellent	Gatherers can collect elements

		incrementally into groups.
Running calculations	✓ Good	Useful when each output depends on previously processed elements.
Custom incremental aggregation	✓ Good	Avoids materializing the complete stream.
Standard mapping/filtering	✗ No	Use map()/filter() instead.
Simple reduction	✗ No	Use reduce() or collect().
Simple grouping	✗ No	Use Collectors.groupingBy().
Complex business workflow	? Usually no	Imperative code may be clearer.
Team unfamiliar with advanced Streams	? Carefully	Gatherers can introduce unnecessary complexity.

Real-Time Use Cases

Use Case	Example	Why Gatherers Help
1. Batch processing	Process transactions in groups of 100.	Gatherer can create batches incrementally.
2. Sliding-window analytics	Calculate the average of the last 5 sensor readings.	Maintains a rolling state while consuming the stream.
3. Log processing	Group consecutive log records by a particular condition.	Stateful processing can identify boundaries between groups.
4. Financial analytics	Calculate running statistics over a transaction stream.	Previous elements can contribute to the next output.
5. IoT processing	Group sensor readings into time/size windows.	Windowing is naturally stateful.
6. Event processing	Detect sequences such as LOGIN → ACTION → LOGOUT.	Gatherer can maintain state across events.
7. Text processing	Group words into	Elements can be accumulated

	paragraphs or fixed-size chunks.	incrementally.
8. Data transformation	Convert a stream into custom domain-specific structures.	Custom gathering logic can be encapsulated and reused.
9. Rate/threshold detection	Emit an output when accumulated values exceed a threshold.	Gatherer can retain an accumulator and emit conditionally.
10. Stream-based parsing	Process tokens while maintaining parser state.	Stateful incremental processing is easier to express.

Example — Batching Transactions

Imagine:

```
List<Transaction> transactions = ...;
```

You want batches of 100:

```
Transaction 1 ┌
Transaction 2 │
...           └─ Batch 1
Transaction 100 ┘
```

```
Transaction 101 ┌
...             └─ Batch 2
Transaction 200 ┘
```

A gatherer allows the pipeline to conceptually become:

```
transactions.stream()
    .gather(batch(100))
    .forEach(transactionBatch -> processBatch(transactionBatch));
```

This is cleaner than repeatedly implementing the same stateful batching logic throughout the application.

Example — Sliding Window

For sensor values:

10, 12, 15, 20, 18

with window size 3:

[10, 12, 15]

[12, 15, 20]

[15, 20, 18]

A gatherer can maintain the rolling window and emit each window as the stream progresses.

This is particularly useful for:

- temperature monitoring
- stock-price analysis
- network metrics
- application performance metrics
- IoT telemetry

Stream Gatherers vs Standard Stream Operations

Requirement	Standard Streams	Stream Gatherers
map()	✓ Excellent	? Overkill
filter()	✓ Excellent	? Overkill
flatMap()	✓ Excellent	? Overkill
Simple sorting	✓ Excellent	✗
Simple grouping	✓ Excellent	✗
Simple reduction	✓ Excellent	?
Fixed-size windows	? Awkward	✓ Excellent
Sliding windows	? Awkward	✓ Excellent
Stateful transformation	? Difficult	✓ Excellent
Custom intermediate operation	✗ Limited	✓ Excellent
Reusable custom stream operator	✗	✓ Excellent

Alternate Features

Alternative	Purpose	When Preferable
map()	One-to-one transformation	When every input produces exactly one output.
filter()	Select elements	When elements only need to be accepted/rejected.
flatMap()	One-to-many transformation	When each element produces another stream of elements.
reduce()	Combine elements into one result	When the final result is a single accumulated value.
collect() / Collector	Mutable reduction into a result container	When you need lists, maps, grouped results, statistics, etc.
takeWhile() / dropWhile()	Conditional stream truncation	When processing depends on a simple boundary condition.
iterate() / generate()	Generate stream elements	When the main problem is producing the stream rather than processing it.
Imperative loop	Explicit stateful processing	Often preferable when custom state makes a stream pipeline difficult to understand.
Reactive Streams libraries	Asynchronous/event-stream processing	Better when the application requires backpressure, asynchronous pipelines, or long-lived event streams.

Stream Gatherer vs Collector

Aspect	Gatherer	Collector
Position in stream	Intermediate operation	Terminal operation
Can produce multiple outputs	✓ Yes	Usually one final result
Stateful processing	✓ Yes	✓

Windowing	✓ Excellent	? Less natural
Running transformation	✓ Excellent	?
Final aggregation	? Possible but not primary purpose	✓ Best
Continue processing afterward	✓ Yes	✗ Terminal operation ends pipeline
Example	.gather(window(5))	.collect(Collectors.toList())

Stream Gatherers vs Alternatives

Requirement	Stream Gatherers	map/filter/flatMap	Collector	Imperative Loop	Reactive Streams
Simple transformation	?	✓ Best	✗	✓	?
Stateful intermediate processing	✓ Best	?	?	✓	✓
Windowing	✓ Best	✗?	?	✓	✓
Final aggregation	?	?	✓ Best	✓	✓
Easy to understand	?	✓ Best	✓	✓	?
Reusable custom stream operation	✓ Excellent	✗	?	✗	✓
Asynchronous processing	✗	✗	✗	✗	✓ Best
Backpressure	✗	✗	✗	✗	✓ Best
Java standard API	✓	✓	✓	✓	✗ Usually external library

Decision Guide

Situation	Recommended
employee -> employee.getName()	map()
Keep employees with salary > ₹50,000	filter()
Convert each order into multiple line items	flatMap()
Calculate total transaction amount	reduce() / collect()
Group transactions by customer	Collectors.groupingBy()
Process every 100 records as a batch	Gatherer
Calculate rolling 5-minute/5-item windows	Gatherer
Create a custom stateful intermediate operation	Gatherer
Complicated multi-step state machine	Consider an imperative loop
Long-running asynchronous event stream	Reactive Streams technology

Key Takeaway

Stream Gatherers (Preview) introduced in Java 22 are designed to fill an important gap in the Stream API:

Standard Stream operations handle common transformations; Gatherers let developers define sophisticated custom intermediate operations.

They are particularly powerful for **windowing, batching, stateful transformations, running calculations, and custom incremental processing.**

However, they should **not replace ordinary Stream operations**. If map(), filter(), flatMap(), reduce(), or collect() expresses the operation clearly, those are usually the better choice.

For a Java 22 project, the practical rule is:

Simple transformation → standard Stream API

Custom stateful intermediate operation → Gatherer

Complex state machine → consider an imperative loop
Asynchronous/backpressure event processing → Reactive Streams

Aspect	Structured Concurrency (Second Preview)
Java Version	Java 22
Year	2024
Feature	Structured Concurrency (Second Preview)
UniqueClassName	StructuredConcurrency
JEP	JEP 462
Status	Second Preview in Java 22
Package	java.util.concurrent
Purpose	Makes concurrent tasks behave as a single unit of work , giving them a clear lifetime and relationship to the task that created them.
Main idea	If a task starts several subtasks, those subtasks should normally finish, fail, or be cancelled within the lifetime of the parent task .
Main API	StructuredTaskScope
Key benefit	Makes concurrent code easier to reason about by establishing a clear parent-child relationship between concurrent tasks.
Relationship with Virtual Threads	Structured Concurrency works especially well with Virtual Threads , but they are separate features. Structured Concurrency manages the <i>structure and lifecycle</i> of concurrent tasks; Virtual Threads provide lightweight threads for executing them.
Typical pattern	java try (var scope = StructuredTaskScope.open()) { var task1 = scope.fork() -> operation1(); var task2 = scope.fork() -> operation2(); scope.join(); return task1.get() + task2.get(); }

Advantages and Disadvantages

Category	Details
Advantage 1 — Clear task hierarchy	Child tasks have an explicit relationship with their parent task, making concurrent execution easier to understand.
Advantage 2 — Better cancellation	If the parent operation fails or is cancelled, related child tasks can be cancelled together.
Advantage 3 — Failure handling	Makes it easier to treat failures among related subtasks as one logical operation.
Advantage 4 — Prevents orphaned tasks	The structured scope helps ensure that child tasks don't accidentally continue running after the parent operation has finished.
Advantage 5 — Easier observability	A task hierarchy makes concurrent operations easier to reason about when diagnosing application behavior.
Advantage 6 — Good fit for concurrent I/O	Works particularly well when multiple independent I/O operations need to be performed concurrently.
Advantage 7 — Works naturally with virtual threads	Lightweight virtual threads make it practical to fork multiple concurrent subtasks without creating a large number of heavyweight platform threads.
Advantage 8 — Cleaner than manually managed executors	Lifecycle and cancellation can be expressed around a lexical scope instead of being scattered across executor-management code.
Disadvantage 1 — Preview feature	In Java 22, it was still a second-preview feature and therefore not appropriate for APIs that require long-term compatibility without accepting preview-feature risk.
Disadvantage 2 — Learning curve	Developers accustomed to <code>ExecutorService</code> , <code>Future</code> , and <code>CompletableFuture</code> need to learn the structured-concurrency model.
Disadvantage 3 — Not suitable for every task	Background jobs intentionally designed to outlive the initiating operation do not naturally fit structured concurrency.
Disadvantage 4 — Scope constraints	Tasks are intentionally tied to the lifetime of the structured scope, which can be restrictive for detached/background work.

Disadvantage 5 — API evolution	As a preview API, details could change in subsequent Java releases.
Disadvantage 6 — Doesn't automatically make code faster	Structured Concurrency primarily improves correctness, lifecycle management, and readability ; performance depends on the workload and execution model.

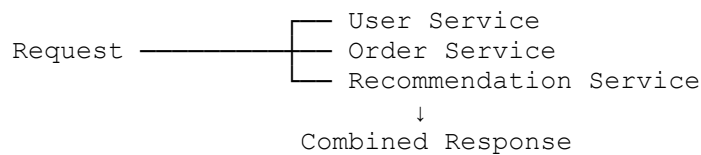
When to Use

Situation	Use?	Reason
Multiple independent service calls	✓ Excellent	Calls can execute concurrently and be treated as one request.
Parallel database/API lookups	✓ Excellent	Parent operation can manage all child operations.
Request-scoped concurrency	✓ Excellent	Child tasks naturally have the same lifetime as the request.
Fan-out/fan-in operations	✓ Excellent	Start multiple tasks, wait for them, combine their results.
Fail-fast operations	✓ Excellent	One failure can trigger cancellation of related tasks.
Race for first successful result	✓ Excellent	Structured task scopes can model "first successful result" patterns.
Short-lived concurrent tasks	✓ Excellent	The lexical scope provides a natural lifetime.
Fire-and-forget background work	✗ Usually no	The work intentionally needs to outlive the parent operation.
Long-running scheduled jobs	✗ Usually no	A scheduler/executor is generally a better abstraction.
Message consumers that run indefinitely	✗ Usually no	Their lifetime is not naturally tied to a short-lived parent scope.
Simple sequential operation	✗ No	Concurrency adds unnecessary complexity.

Existing CompletableFuture pipeline that works well	❓	Migration is not automatically necessary; evaluate readability and lifecycle requirements.
--	---	--

Real-Time Use Case 1 — Microservice Aggregation

Suppose an API endpoint needs data from three independent services:



Instead of calling them sequentially:

```

User      → 100 ms
Orders    → 150 ms
Recommend → 200 ms
  
```

Total $\approx$ 450 ms

they can execute concurrently:

```

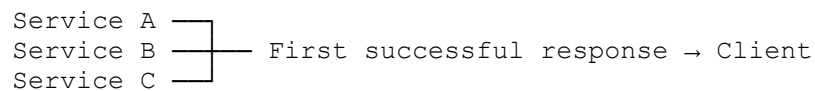
User      _____ 100 ms
Orders    _____ 150 ms
Recommend _____ 200 ms
  
```

Total $\approx$ 200 ms + overhead

Structured Concurrency makes those child operations part of the same request-scoped task.

Real-Time Use Case 2 — First Successful Response

Suppose an application queries multiple redundant services:



If Service B responds successfully first, the remaining operations can be cancelled.

This is useful for:

- redundant service endpoints
- geographically distributed services
- fallback providers
- replicated data sources

Real-Time Use Case 3 — Parallel Database Queries

An analytics endpoint might need:

Customer information
+
Transaction history
+
Account summary
↓
Combined dashboard

These independent queries can execute concurrently within one structured scope.

Real-Time Use Case 4 — E-Commerce Product Page

A product page may need:

Product details
├— Inventory
├— Pricing
├— Reviews
└— Recommendations

Instead of performing each call sequentially, independent operations can run concurrently and be joined before producing the response.

Real-Time Use Case 5 — Travel Search

A travel application may query several providers:

Airline A ┌
Airline B ┤ — Search results
Airline C ┤
Hotel API └

Structured Concurrency is useful when all these subtasks belong to one user search operation.

Example

A simplified conceptual example:

```
import java.util.concurrent.StructuredTaskScope;

class DashboardService {
```

Dashboard loadDashboard() throws Exception {

```
    try (var scope = StructuredTaskScope.open()) {
```

```
        var user = scope.fork(this::loadUser);
```

```
        var orders = scope.fork(this::loadOrders);
```

```
        var recommendations = scope.fork(this::loadRecommendations);
```

```
        scope.join();
```

```
        return new Dashboard(
```

```
            user.get(),
```

```
            orders.get(),
```

```
            recommendations.get()
```

```
        );
```

```
    }
```

```
}
```

```
User loadUser() {
```

```
    return ...;
```

```
}
```

```
Orders loadOrders() {
```

```
    return ...;
```

```
}
```

```
Recommendations loadRecommendations() {
```

```
    return ...;
```

```
}
```

```
}
```

The important concept is:

```
loadDashboard()
```

```
|
```

```
├── loadUser()
```

```
├── loadOrders()
```

```
└── loadRecommendations()
```

```
↓
```

```
scope.join()
```

```
↓
```

```
Combined result
```

The child tasks are **structured under the parent operation**.

Structured Concurrency vs Traditional Approaches

Aspect	Structured Concurrency	ExecutorService	Completable Future	Virtual Threads Alone
Parent-child task relationship	✓ Strong	Ⓜ Manual	Ⓜ Manual	✗
Structured lifecycle	✓ Yes	Ⓜ Manual	Ⓜ Manual	✗
Cancellation propagation	✓ Built into model	Ⓜ Manual	Ⓜ Manual	✗ Automatic
Concurrent I/O	✓ Excellent	✓ Excellent	✓ Excellent	✓ Excellent
Async composition	Ⓜ Not primary purpose	✗	✓ Excellent	✗
Simple concurrent tasks	✓ Excellent	✓	Ⓜ More verbose	✓
Fire-and-forget	✗	✓	✓	✓
Works with virtual threads	✓ Yes	✓	✓	—
Main purpose	Concurrency structure/lifecycle	Task execution	Async composition	Lightweight threads

Alternate Features

Alternative	Description	When Preferable
Virtual Threads	Lightweight threads designed for high-concurrency applications, especially blocking I/O	When the primary need is handling many concurrent tasks efficiently.
ExecutorService	Traditional task execution framework	When you need explicit executor lifecycle, queues, scheduling, or compatibility with existing code.

CompletableFuture	Asynchronous computation and composition	When you need complex asynchronous pipelines, callbacks, and non-lexical composition.
Future	Represents the result of an asynchronous computation	Suitable for simpler task submission and result retrieval.
ForkJoinPool	Work-stealing task execution	Useful for CPU-bound parallel computation.
Parallel Streams	Declarative data parallelism	Useful for independent operations over collections, rather than request/task orchestration.
Reactive Streams	Asynchronous streams with backpressure	Prefer for long-lived event streams and reactive pipelines.

Structured Concurrency vs Alternatives

Requirement	Structured Concurrency	Virtual Threads	Completable Future	ExecutorService	Reactive Streams
Parent-child lifecycle	✓ Excellent	✗	?	?	?
Cancellation propagation	✓ Excellent	✗	?	?	✓
Blocking I/O	✓ Excellent	✓ Excellent	✓	✓	?
Async pipelines	?	✗	✓ Best	?	✓
Fire-and-forget	✗	✓	✓	✓	✓
Long-running streams	✗	?	?	?	✓ Best
Simple concurrent request	✓ Excellent	✓	?	?	✗
CPU	?	?	?	✓	?

parallelism					
Code readability for scoped tasks	✓ Excellent	✓	?	?	?
Java 22 status	Second Preview	Final	Stable	Stable	Depends on implementation

Decision Guide

Scenario	Recommended
Multiple API calls belonging to one request	Structured Concurrency
Multiple database calls belonging to one request	Structured Concurrency
Need lightweight threads for many blocking operations	Virtual Threads
Need complex asynchronous callback pipeline	CompletableFuture
Need explicit thread-pool control	ExecutorService
Need long-lived event processing/backpressure	Reactive Streams
Need CPU-bound parallel computation	ForkJoinPool / parallelism
Fire-and-forget background task	ExecutorService / scheduler

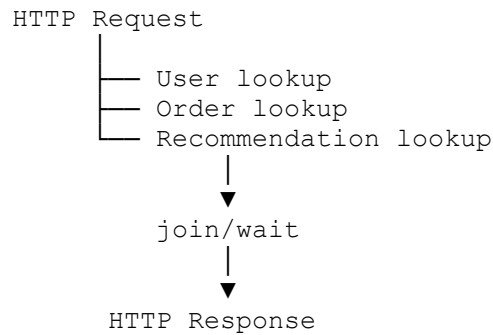
Key Takeaway

Structured Concurrency (Second Preview) is primarily about **organizing concurrent work**, not simply making Java programs faster.

Its central principle is:

If several tasks belong to one operation, their lifetimes should be structured around that operation.

For example:



This makes **cancellation, failure handling, task lifetime, and observability** much easier to reason about.

For Java 22, remember that **Structured Concurrency was still a preview feature**, so production adoption should account for preview-feature compatibility. For new Java applications, it is especially compelling when combined with **Virtual Threads** for request-scoped, concurrent I/O operations.

Aspect	Implicitly Declared Classes and Instance Main Methods (Second Preview)
Java Version	Java 22
Year	2024
Feature	Implicitly Declared Classes and Instance Main Methods (Second Preview)
UniqueClassName	ImplicitClassesInstanceMain
JEP	JEP 463
Status	Second Preview in Java 22
Main goal	Make Java easier for beginners and small programs by allowing source code to be written without requiring the traditional explicit class declaration and public static void main(String[] args) boilerplate.

Core idea	The Java compiler can implicitly provide a class around a collection of top-level statements/methods, while an instance main method can serve as the program entry point.
Traditional Java	java public class Hello { public static void main(String[] args) { System.out.println("Hello"); } }
Simplified style	java void main() { System.out.println("Hello"); }
Implicit class	The developer does not need to explicitly write class Hello { ... } for a simple source program.
Instance main	main does not have to be static; the Java launcher can create an instance of the implicitly declared class and invoke the instance main method.
Target audience	Primarily beginners, educators, small scripts/programs, and developers experimenting with Java.

Advantages and Disadvantages

Category	Details
Advantage 1 — Less boilerplate	Removes repetitive class and static main syntax from simple programs.
Advantage 2 — Beginner friendly	Students can focus on Java statements and concepts before learning all of the ceremony around classes and static methods.
Advantage 3 — Faster experimentation	Small programs can be written and executed quickly.
Advantage 4 — More approachable Java	Java can look more like a simple scripting language for introductory programs.
Advantage 5 — Instance-based programming	Allows introductory programs to use instance methods and fields without immediately requiring static concepts.
Advantage 6 — Natural progression	Learners can start with simple source code and gradually introduce explicit classes and more advanced Java concepts.
Advantage 7 — Good for teaching	Teachers can introduce variables, control flow, methods, and basic APIs before explaining class

	boilerplate.
Disadvantage 1 — Preview feature	In Java 22 it was still a second-preview feature and required preview support.
Disadvantage 2 — Potential confusion	Beginners may initially be confused when they later encounter the traditional explicit class and static main model.
Disadvantage 3 — Limited fit for large applications	Enterprise applications benefit from explicit classes, packages, modules, and established project structures.
Disadvantage 4 — Tooling considerations	Some tools, educational environments, static analyzers, or older IDE configurations may not fully support preview syntax.
Disadvantage 5 — Migration considerations	Code using the preview syntax may need changes as the feature evolves.
Disadvantage 6 — Less explicit structure	Hiding the class declaration is convenient for small programs but can reduce clarity when the source becomes large.
Disadvantage 7 — Not a performance feature	It primarily changes the programming experience and entry-point model; it does not inherently make programs execute faster.

Traditional Java vs Java 22 Preview

Aspect	Traditional Java	Implicit Classes + Instance main
Explicit class declaration	Required	✗ Can be implicit
main must be static	Traditionally yes	✗ Not necessarily
public static void main(String[] args)	Common entry point	Simpler forms supported
Boilerplate	Higher	Lower
Beginner friendly	?	✓ Excellent
Large application structure	✓ Excellent	?

Preview required in Java 22	✗	✓
Production-library suitability	✓	?
Teaching introductory Java	?	✓ Excellent

When to Use

Situation	Use?	Reason
Learning Java	✓ Excellent	Students can focus on programming concepts instead of boilerplate.
Teaching programming	✓ Excellent	Makes first examples significantly shorter.
Small command-line program	✓ Good	Eliminates unnecessary ceremony.
Coding exercises	✓ Excellent	Useful for short standalone examples.
Algorithm demonstrations	✓ Good	Keeps the focus on the algorithm.
Small experimentation/prototyping	✓ Good	Quick to write and run.
Large enterprise application	✗ No	Explicit application structure is preferable.
Reusable library	✗ No	Explicit classes and APIs provide clearer structure.
Public framework	✗ No	Explicit types and APIs are more appropriate.
Long-lived production codebase	? Usually no	Preview feature and implicit structure may not provide enough benefit.
Application with many classes/packages	✗ No	Explicit class/package organization is clearer.

Real-Time Use Cases

Use Case	Example	Benefit
1. Beginner's first program	java void main() { System.out.println("Hello World"); } 	Removes class and static boilerplate.
2. Simple calculator	java void main() { int a = 10; int b = 20; System.out.println(a + b); } 	Students can focus on variables/operators.
3. Algorithm demonstration	Write sorting/searching logic directly in a simple source file.	Keeps educational examples compact.
4. Coding interview practice	Quickly implement and execute a small algorithm.	Reduces setup code.
5. Command-line utility	A small file-processing or conversion program.	Useful when a full class structure adds little value.
6. Teaching OOP progression	Start with an instance main, then introduce explicit classes.	Provides a gradual learning path.
7. API experimentation	Test a JDK API with a small source file.	Faster experimentation.
8. Workshop/tutorial examples	Demonstrate loops, collections, streams, or APIs.	More code can be dedicated to the concept being taught.

Example — Simple Program

Traditional Java:

```
public class Calculator {  
  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 20;  
  
        System.out.println("Sum = " + (a + b));  
    }  
}
```

With the Java 22 preview feature:

```
void main() {  
    int a = 10;  
    int b = 20;  
  
    System.out.println("Sum = " + (a + b));  
}
```

The second form removes the ceremony that is not important to someone learning basic programming.

Example — Instance State

One interesting aspect is that the entry point can be an **instance method**:

```
class-style concept {  
    int counter = 10;  
  
    void main() {  
        System.out.println(counter);  
    }  
}
```

The important conceptual difference is that the program can operate in an **object context** rather than requiring the traditional static entry point.

Alternate Features

Alternative	Purpose	When Preferable
Traditional public static void main	Standard Java application entry point	Best for production and conventional applications
Single-file source launcher	Run a source file directly with java File.java	Small programs and scripts
JShell	Interactive Java experimentation	When you want a REPL rather than a source program
Explicit class declaration	Provides clear object/type structure	Medium and large applications
Maven/Gradle project	Full build/dependency/project management	Enterprise and production applications

Unit test methods	Execute focused pieces of code as tests	Algorithm experimentation and automated testing
--------------------------	---	---

Implicit Classes vs Alternatives

Requirement	Implicit Classes + Instance main	Traditional main	JShell	Single-File Launcher
Minimal boilerplate	✓ Excellent	✗	✓ Excellent	✓
Explicit class required	✗	✓	✗	Usually
Instance main	✓	✗	N/A	Depends
Beginner friendly	✓ Excellent	?	✓ Excellent	✓
Interactive experimentation	?	✗	✓ Best	?
Small executable program	✓ Excellent	✓	?	✓
Large application	✗	✓ Best	✗	✗
Production compatibility	? Preview in Java 22	✓ Best	✗	?
Clear type structure	?	✓ Excellent	✗	?
Suitable for teaching	✓ Excellent	✓	✓ Excellent	✓

Decision Guide

Situation	Recommended
First Java program	Implicit class + instance main
Teaching basic Java syntax	Implicit class + instance main
Quick small program	Implicit class + instance main or single-file launcher

Interactive experimentation	JShell
Small production command-line application	Traditional explicit class
Enterprise application	Traditional explicit classes + Maven/Gradle
Library development	Explicit classes/interfaces
Public API	Explicit classes/interfaces
Long-term production compatibility	Traditional public static void main

Key Takeaway

Implicitly Declared Classes and Instance Main Methods (Second Preview) is primarily a **Java learning and simplicity feature**, not a performance or concurrency feature.

Its goal can be summarized as:

Let beginners write useful Java programs before they need to understand all of Java's traditional class and static main ceremony.

For example:

```
void main() {
    System.out.println("Hello Java 22!");
}
```

is much easier for a beginner to understand than:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello Java 22!");
    }
}
```

However, for **large production systems, libraries, frameworks, and long-lived codebases**, explicit classes and the conventional entry-point structure remain preferable.

Aspect	Scoped Values (Second Preview)
Java Version	Java 22
Year	2024
Feature	Scoped Values (Second Preview)
UniqueClassName	ScopedValues
JEP	JEP 464
Status	Second Preview in Java 22
Package	java.lang
Purpose	Provides a mechanism for safely and efficiently sharing immutable data with methods and child tasks for a bounded period of execution.
Main API	ScopedValue<T>
Core concept	A value is bound to a scope and can be accessed by methods executing within that scope.
Simple idea	Instead of passing the same context value through many method parameters, a scoped value can make that value available to deeply nested methods without explicitly passing it through every method call.
Important property	Scoped values are immutable from the point of view of the code receiving them . A binding can be established for a scope, but ordinary code cannot simply modify the bound value.
Typical data	Request context, security context, transaction context, tracing information, tenant ID, locale, correlation ID, etc.

Advantages and Disadvantages

Category	Details
Advantage 1 — Avoids parameter plumbing	Context information can be accessed by deeply nested methods without adding the same parameter to every method signature.
Advantage 2 — Immutable context	A scoped value is designed for sharing data that

	should not be modified by callees.
Advantage 3 — Clear lifetime	A binding exists only within a defined lexical/dynamic scope and automatically becomes unavailable outside it.
Advantage 4 — Good for request context	Request-specific information such as correlation IDs or tenant information can be made available throughout request processing.
Advantage 5 — Works well with structured concurrency	Scoped Values are particularly useful for propagating context into structured concurrent subtasks.
Advantage 6 — Better alternative to some ThreadLocal uses	Scoped Values are designed for one-way context sharing rather than mutable thread-associated state.
Advantage 7 — Efficient with virtual threads	The design is well suited to Java's modern concurrency model, including large numbers of virtual threads.
Advantage 8 — Easier reasoning	Because values are immutable and scoped, accidental modification and uncontrolled lifetime are reduced.
Disadvantage 1 — Preview feature	In Java 22 it was still a second-preview feature and required preview support.
Disadvantage 2 — Not a general replacement for parameters	If a value is fundamental to a method's business contract, passing it explicitly is often clearer.
Disadvantage 3 — Context can become hidden	A method may depend on a scoped value without that dependency appearing in its parameter list.
Disadvantage 4 — Requires understanding scope	Developers must understand binding, inheritance, lifetime, and execution context.
Disadvantage 5 — Poor fit for mutable shared state	Scoped Values are designed around immutable bindings, not mutable state shared between unrelated tasks.
Disadvantage 6 — Migration complexity	Replacing an established ThreadLocal architecture can require significant design changes.
Disadvantage 7 — API evolution risk	As a preview API, its details could change in later Java releases.

--	--

Basic Example

A simplified example:

```
static final ScopedValue<String> USER =
    ScopedValue.newInstance();

static void processRequest() {
    System.out.println("User: " + USER.get());
}

public static void main(String[] args) {

    ScopedValue.where(USER, "Anurag")
        .run() -> {
            processRequest();
        };
}
```

Conceptually:

```
main()
├── bind USER = "Anurag"
│   ├── processRequest()
│   │   └── USER.get()
│   └── other methods()
```

The binding exists only inside the scope established by `where(...).run(...)`.

When to Use

Situation	Use?	Reason
Request context	✓ Excellent	Request metadata can be available throughout processing.
Correlation/trace ID	✓ Excellent	Logging/tracing code can access the ID without parameter plumbing.
Security/user context	✓ Good	Immutable security information can

		be shared within a request scope.
Tenant ID	✓ Excellent	Multi-tenant applications can make the current tenant available to nested operations.
Locale/context settings	✓ Good	Context can be propagated to nested operations.
Structured concurrent tasks	✓ Excellent	Context can be shared with related child tasks.
Virtual-thread request processing	✓ Excellent	Fits the modern lightweight-thread model.
Mutable global state	✗ No	Scoped Values are not intended to replace mutable shared state.
Simple method argument	✗ Usually no	Explicit parameters are clearer.
Long-lived application state	✗ No	Use appropriate application/service state management.
Caching	✗ No	Use a proper cache.
Cross-request global configuration	✗ No	Use configuration/application-level mechanisms.

Real-Time Use Case 1 — Request Correlation ID

In a microservice:

```

HTTP Request
├── correlationId = "REQ-12345"
│   ├── Controller
│   ├── Service
│   ├── Repository
│   └── Logging

```

Instead of passing:

```

service.process(request, correlationId);
repository.save(data, correlationId);
logger.log(message, correlationId);

```

each method can access the scoped correlation ID when appropriate.

```
static final ScopedValue<String> CORRELATION_ID =  
    ScopedValue.newInstance();  
  
static void handleRequest() {  
    ScopedValue.where(CORRELATION_ID, "REQ-12345")  
        .run() -> process();  
}  
  
static void process() {  
    log("Processing request");  
}  
  
static void log(String message) {  
    System.out.println(  
        CORRELATION_ID.get() + " : " + message  
    );  
}
```

Real-Time Use Case 2 — Multi-Tenant Application

A SaaS application can establish:

```
TENANT_ID = "tenant-42"
```

at the beginning of request processing.

Deeply nested code can access the current tenant without adding tenantId to every method signature.

```
static final ScopedValue<String> TENANT =  
    ScopedValue.newInstance();  
  
ScopedValue.where(TENANT, "tenant-42")  
    .run() -> billingService.process();
```

This is particularly useful when many layers need the same immutable request context.

Real-Time Use Case 3 — Security Context

A request can establish an immutable security context:

```
record SecurityContext(String user, String role) {}
```

```
static final ScopedValue<SecurityContext> SECURITY =  
    ScopedValue.newInstance();
```

```

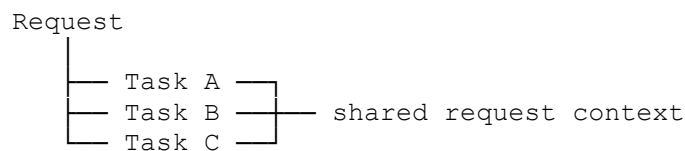
ScopedValue.where(
    SECURITY,
    new SecurityContext("anurag", "ADMIN")
).run() -> {
    performOperation();
};

```

Nested authorization code can inspect the current context.

Real-Time Use Case 4 — Structured Concurrency

Scoped Values become particularly interesting when combined with structured concurrency:



For example, all child tasks can access a request's correlation ID or tenant ID while remaining within the parent operation's scope.

Real-Time Use Case 5 — Distributed Tracing

A tracing system can bind:

```

traceId = "abc-123"
spanId = "span-456"

```

at the start of an operation.

Logging and tracing code deep in the call stack can retrieve the current context without requiring every method to receive trace identifiers as parameters.

Scoped Values vs ThreadLocal

Aspect	ScopedValue	ThreadLocal
Main purpose	Immutable scoped context	Thread-associated state
Mutability	Designed for immutable	Value can be changed

	bindings	
Lifetime	Bounded scope	Can persist until explicitly removed/thread ends
Explicit cleanup	Usually unnecessary after scope	remove() often important
Data flow	Context flows into called code	Associated with current thread
Child-task propagation	Designed to work with structured concurrency	More cumbersome
Virtual-thread suitability	Excellent	Can be used, but has different lifecycle/resource considerations
Accidental state leakage	Lower due to bounded scope	Higher if cleanup is forgotten
Mutable state	✗ Not the purpose	✓ Possible
Request context	✓ Excellent	✓ Common
Legacy compatibility	Newer API	Very mature

Alternate Features

Alternative	Purpose	When Preferable
Method parameters	Explicitly pass context through the call chain	Best when the context is an important part of the method's business contract.
ThreadLocal	Associate data with the current thread	Best for existing applications that already use thread-local context or genuinely require mutable thread-associated state.
Context object	Bundle request/application context into one object	Good when several pieces of context need to be passed explicitly.
Dependency Injection	Supply contextual dependencies through	Useful in Spring/CDI-style enterprise applications.

	framework-managed components	
Request-scoped objects	Associate context with a request lifecycle	Useful in web frameworks.
Structured Concurrency	Manage child-task lifetimes	Best when the problem is task hierarchy; Scoped Values can complement it for context propagation.

Scoped Values vs Alternatives

Requirement	Scoped Values	Method Parameters	ThreadLocal	DI/Request Scope
Explicit data dependency	?	✓ Best	✗	?
Avoid parameter plumbing	✓ Excellent	✗	✓	✓
Immutable context	✓ Excellent	✓	?	Depends
Bounded lifetime	✓ Excellent	Depends	?	✓
Mutable state	✗	✓	✓	Depends
Request context	✓ Excellent	?	✓	✓ Excellent
Structured concurrency	✓ Excellent	?	?	?
Virtual-thread applications	✓ Excellent	✓	?	Depends
Simple code	?	✓ Best	?	?
Framework integration	?	✓	✓	✓ Best
Java standard API	✓	✓	✓	Framework dependent

Decision Guide

Scenario	Recommended
Value is fundamental to method's business logic	Method parameter
Immutable request context shared deeply	Scoped Value
Correlation/trace ID	Scoped Value
Tenant ID for a request	Scoped Value
Immutable security context	Scoped Value
Existing mutable thread-local state	ThreadLocal
Dependency managed by Spring/CDI	Dependency Injection / request scope
Multiple concurrent child tasks need request context	Scoped Value + Structured Concurrency
Global application configuration	Configuration object/service
Cache	Dedicated cache

Key Takeaway

Scoped Values (Second Preview) is primarily a **context-propagation** feature.

Its central idea is:

Make immutable contextual data available to nested operations for a well-defined period without passing it through every method parameter.

The strongest use cases are:

- request correlation IDs
- distributed tracing
- tenant context
- security context
- request-scoped configuration
- context propagation across structured concurrent tasks

The most important distinction is:

Method parameter → explicit business dependency

Scoped Value → implicit, immutable execution context

ThreadLocal → mutable thread-associated state

For Java 22, remember that Scoped Values were still a **second-preview feature**, so they should be evaluated with preview-feature compatibility in mind rather than treated as a finalized Java API.